

TaniOS Enterprise Architecture Guide

Full-Stack Technical Specification & Scaling Blueprint | Class-leading AI Learning Platform

1. SYSTEM CONTEXT & SERVERLESS TOPOLOGY

TaniOS is built upon an ultra-modern Serverless & BaaS (Backend-as-a-Service) topology. Traditional VM servers require perpetual configuration, OS patching, load balancers, and constant database optimization. In contrast, TaniOS handles core authentication and standard read/write operations securely by talking directly to Firebase BaaS, and delegates high-intensity AI operations to Vercel Serverless cloud runtime execution.

Structural Roles:

- **Vite React Client** Renders highly responsive interactive frontends, synchronizes streaks, compiles learning weaknesses, post-processes math formulas into mathematical LaTeX, and tracks client state.
- **Vercel Serverless Gateways** An on-demand API gateway inside 'api/generate.js'. Contains strict IP sliding window rate limit monitors, handles keys/model parameters rotation dynamically if a key quota runs out, races prompts against a 35-second abort timer, and normalizes serverless exceptions.
- **Firebase Infrastructure BaaS** Provides secure out-of-the-box user management, stores chat history collections, and processes client requests securely through Firestore database rules without any intermediate backend API code.

2. FRONTEND LIFECYCLE & UTILITIES

The client application functions as a compiled Single-Page Application (SPA) maximizing state isolation. Standard directories contain isolated utilities that guard performance:

- **rateLimiter.js** Enforces client-side request pooling. Restricts local triggers to a max of 10 calls per minute in real time, preventing duplicate button clicks or loops from hitting the cloud endpoint.
- **ai.js**: Acts as the unified interface to the server backend. Packages inputs and manages the state of the generator outputs. Integrates `fixMathFormatting()` which uses regular expression patterns to dynamically rewrite raw unicode characters, fractions, and Greek notation into clean LaTeX strings for KaTeX components.
- **AuthContext.js** A React context wrapping the app directory. Handles student states, tokens, guest-mode checks, and hydrates dashboard XP states securely on reload.

3. BACKEND GATEWAY SPECIFICATION (api/generate.js)

The server functions as a highly scalable cloud function. Key modules include:

- **IP Sliding Window Rate Limiter** Saves timestamps per unique remote IP address. Automatically purges dead tracking records when active maps exceed 2000 entries to guarantee zero serverless memory overhead. Rejects spammers with a clear 429 status code.
- **Rotating API Keys & Models Pipeline** Parses GEMINI_API_KEYS dynamically. If a key triggers a 429 rate limit, 503 service overload, quota exhaustion, or timing out, it rotates to the next key index in real time. It also falls back gracefully from

gemini-2.5-flash to gemini-2.5-pro to maximize availability.

- **Timeout Guard** Employs `Promise.race()` running a strict 35s timer. Aborts calls if the AI model hangs, preserving serverless execution limits and rotating the call instantly.
- **Error Normalizer** Transforms scary tracebacks into supportive student messages, keeping full crash analytics packed inside an invisible private diagnostics response object for developer logging.

4. DATABASE SEGREGATION & RULES

Security is enforced at the database layer using strict Firestore Security Rules. User records, chat histories, note collections, and custom timetable sheets are securely locked down to prevent cross-tenant access. Students can only read or write directories where firestore credentials exactly match their authenticated UID context.

5. PRODUCTION SCALING PLAN

- **Phase 1: Present** Firebase BaaS coupled with Vercel API. Infinite auto-scaling with \$0 idle monthly maintenance costs.
- **Phase 2: High Scale** Integrate Upstash Redis Caching on serverless calls to intercept redundant popular inquiries, reducing API tokens. Migrate functions to Vercel V8 Edge Runtimes to completely eliminate serverless cold starts.
- **Phase 3: Mature Enterprise** Migrate serverless node logic into dedicated Dockerized Express microservices running on AWS ECS or GCP Cloud Run, connecting directly to robust relational PostgreSQL database instances.